EC2 Access Recovery Guide

How to regain SSH access when your PEM key is lost

Ubuntu Laptop → Ubuntu EC2 Instance

Overview

This guide walks you through recovering SSH access to an AWS EC2 instance when you have lost or accidentally deleted your PEM key file. It covers the most reliable method: using EC2 Instance Connect (browser terminal) to manually inject a new public key.

Prerequisites

- An AWS account with access to the EC2 Console
- The EC2 instance must have EC2 Instance Connect enabled (default on Ubuntu AMIs)
- Ubuntu laptop with terminal access
- The instance should be in Running state

Available Recovery Methods

Method	When to Use
EC2 Instance Connect (Recommended)	Instance is running, Ubuntu AMI — fastest and most reliable
User Data Script	Instance is stopped — injects key on next boot
SSM Session Manager	SSM Agent is pre-installed on the instance
EBS Volume Detach	Last resort — instance is stopped, other methods unavailable

Recommended Method: EC2 Instance Connect + Manual Key Injection

This is the fastest and most reliable method. It uses the AWS browser-based terminal to directly add your new public key without any reboot required.

Step 1 — Generate a New SSH Key Pair (No Passphrase)

Open your terminal on your Ubuntu laptop and run:

```
ssh-keygen -t rsa -b 4096 -f ~/my-new-key -N ""
```

The -N "" flag sets an empty passphrase so SSH never prompts you for one.

This creates two files:

- ~/my-new-key — your private key (keep this safe!)
- ~/my-new-key.pub — your public key (this gets added to the server)

Copy your public key by running:

```
cat ~/my-new-key.pub
```

Select and copy the entire output. It starts with ssh-rsa AAAA... and ends with your username@hostname.

Step 2 — Open Browser Terminal via EC2 Instance Connect

1. Go to AWS Console → EC2 → Instances
2. Select your instance
3. Click the Connect button at the top
4. Choose the EC2 Instance Connect tab
5. Click Connect — a browser terminal opens directly into your instance

NOTE: EC2 Instance Connect works without any key — AWS pushes a temporary one-time key valid for 60 seconds to open the session.

Step 3 — Add Your New Public Key via Browser Terminal

In the browser terminal, run these commands one by one:

Create .ssh directory if it doesn't exist:

```
mkdir -p /home/ubuntu/.ssh
```

Add your public key (replace the key below with your own):

```
echo "ssh-rsa AAAA...your-full-public-key... user@host" >>
/home/ubuntu/.ssh/authorized_keys
```

Set correct permissions:

```
chmod 700 /home/ubuntu/.ssh
chmod 600 /home/ubuntu/.ssh/authorized_keys
chown -R ubuntu:ubuntu /home/ubuntu/.ssh
```

Verify the key was added successfully:

```
cat /home/ubuntu/.ssh/authorized_keys
```

TIP: You should see your ssh-rsa AAAA... key printed in the output. If the file is empty, repeat the echo command above.

Step 4 — SSH From Your Laptop

Back in your local terminal, first set the correct permission on your private key:

```
chmod 400 ~/my-new-key
```

Then connect to your instance:

```
ssh -i ~/my-new-key ubuntu@YOUR_PUBLIC_IP
```

Replace YOUR_PUBLIC_IP with the Public IPv4 address shown in your EC2 Console.

SUCCESS: You should now be logged into your instance without any passphrase prompt!

Alternate Method: User Data Script (Instance Must Be Stopped)

Use this method if EC2 Instance Connect is not available or disabled on your instance.

Step 1 — Stop the Instance

Go to EC2 Console → Instance State → Stop and wait for it to fully stop.

Step 2 — Edit User Data

6. Go to Actions → Instance Settings → Edit User Data
7. Clear any existing content
8. Paste the script below, replacing the key with your own public key:

```
#!/bin/bash
mkdir -p /home/ubuntu/.ssh
echo "ssh-rsa AAAA...your-key..." >> /home/ubuntu/.ssh/authorized_keys
chmod 700 /home/ubuntu/.ssh
chmod 600 /home/ubuntu/.ssh/authorized_keys
chown -R ubuntu:ubuntu /home/ubuntu/.ssh
```

IMPORTANT: Do NOT include the Content-Type header line — it can prevent the script from executing on some Ubuntu AMIs.

Step 3 — Start the Instance and SSH

9. Save the User Data
10. Start the instance and wait for both status checks to turn green
11. SSH using: ssh -i ~/my-new-key ubuntu@YOUR_PUBLIC_IP

Troubleshooting

Error	Fix
Permission denied (publickey)	Key not added correctly — verify with cat ~/.ssh/authorized_keys
Enter passphrase for key	Regenerate key with -N "" flag for no passphrase
Connection timed out	Check Security Group — port 22 must be open
WARNING: UNPROTECTED KEY	Run: chmod 400 ~/my-new-key
User Data script didn't run	Stop instance fully, clear old User Data, re-paste and start again

Best Practices to Avoid This in Future

- Always back up your PEM key to a secure location (password manager, encrypted drive)
- Enable EC2 Instance Connect on all instances — it's your emergency access fallback
- Consider using SSM Session Manager — no keys needed at all
- When generating keys, use -N "" for no passphrase unless security requires it
- Store key files outside the Downloads folder — use ~/.ssh/ directory

PRO TIP: Move keys to ~/.ssh/ with: mv ~/my-new-key ~/.ssh/ && chmod 400 ~/.ssh/my-new-key — this is the standard location SSH expects.

Quick Reference Commands

Action	Command
Generate key (no passphrase)	ssh-keygen -t rsa -b 4096 -f ~/my-new-key -N ""
View public key	cat ~/my-new-key.pub
Set key permission	chmod 400 ~/my-new-key
SSH into instance	ssh -i ~/my-new-key ubuntu@YOUR_IP

Verify authorized_keys	cat /home/ubuntu/.ssh/authorized_keys
Fix permissions on server	chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys